Sistema distribuido para la resolución de problemas SAT

Memoria

Universitat de Lleida
Escola Politécnica Superior
Grau en Enginyeria Informática

Autor:
Jose Miguel Avellana López
Codirector:
Josep Luis Lérida Monsó
Codirector:
Josep Argelich Romá

1 Introdución

1.1 Contexto y motivación

En 1970, el matemático británico John Horton Conway diseño un automata celular al que llamo El Juego de la Vida. Un autómata celular es un modelo matemático y computacional para un sistema dinámico que evoluciona en pasos discretos.
El juego de la vida viene a decir que, a partir de un conjunto de reglas simples, se pueden generar estructuras complejas. Un sistema compuesto de un conjunto de agentes (o células), que ocupan un lugar en él, que poseen una serie de reglas que definen como interaccionan con el resto de agentes.
Y es que, en cierto modo, asi es como funciona la evolución de un ecosistema de seres vivos, mejorando y optimizando sus procesos sin que exista una intención consciente de ello por parte de sus agentes.
Bajo este mismo principio se puede entender la capacidad de los sistemas económicos basados en la competencia para mejorar sus procesos. Cuando una empresa se dedica a investigar nuevas tecnologías para abaratar costes, no lo hace por el bien común, lo hace para vencer a sus rivales, pero, de manera inconsciente, sirve al bien común. Será el mercado el que dictará si esta nueva forma de realizar dicha tarea es más optima o no.
Por lo tanto, es la competencia de ideas, de procesos y formas de actuación la que nos permite mejorar la forma en que vivimos. Teniendo en mente, por ejemplo, procesos industriales o logísticos, si a alguien se le ocurre una forma más eficiente de hacer algo creará una empresa que realice esta labor en la manera que propone, si realmente es mejor que el método tradicional, terminará abarcando un cada vez mayor porcentaje de su mercado e imponiendose al método anterior.
Pero esta es una forma muy lenta de evolucionar debido a que el tiempo para observar resultados es el de un escenario real. Sin embargo, podemos observar nuevas propuestas de actuación en entornos simulados, en mucho menor tiempo y barajando más opciones, y de esta forma aplicar la mejor forma de actuación practicamente al instante.
La revolución informática de las ultimas décadas nos da la posibilidad de simular y automatizar procesos. Hemos construido una red de ordenadores con estas capacidades, y su potencial es enorme. Tecnologías de vanguardia como la inteligencia artificial nos prometen una significativa mejora en nuestra calidad de vida, principalmente por su potencial para automatizar tareas como nunca antes se había imaginado.
En un automata celular, a menor número y/o complejidad de las reglas sobre la interacción de sus agentes mayor será el espectro de complejidad que pueda alcanzar el autómata. Contra más reglas le pones más se limita. ¿?
Por ello considero interesante un sistema distribuido de servicios. Desde mi punto de vista, existen dos componentes clave:

- Un servicio es un contendor de software que realiza una tarea especifica, puede entenderse como una "caja negra", existe una especificación del mismo en formato binario, donde se especifica todo lo necesario para construirlo y comunicarse con él.

- Un nodo es un computador que forma parte de la red, debe de tener la capacidad de encontrar otros nodos y comunicarse con ellos, compartir y construir servicios y delegar la sobrecarga de servicios a sus pares.

1.2 Objetivos del proyecto

Este trabajo pretende dos cosas:

- Definir, de manera simple, las reglas básicas de cada uno de los componentes de un sistema distribuido de servicios, y como se definen e interaccionan entre sí.

- Dar uso del sistema para realizar una prueba concepto sobre un problema concreto, la resolución de problemas SAT.

Más concretamente, los objetivos son (algunos conceptos se desarrollan más adelante):

- Diseño de la especificación de un servicio.
- Desarrollo de un nodo con capacidad para:
  - Construir y ejecutar servicios.
  - Compartir servicios con el resto de los nodos de la red.
  - Balancear la carga de servicios entre sí mismo y el resto de nodos de la red.
- Desarrollo de un servicio de clasificación y selección de servicios que resuelven SAT. Este debe ser capaz de:
  - Cargar especificaciones de SAT-solvers, pedirle al nodo que le devuelva instancias de los mismos, mantenerlos (comprobar que siguen activos).
  - Clasificar los SAT-solvers que se le han especificado.
  - Seleccionar un SAT-solver en función de un CNF dado.

2 "State of the art"

Dentro de un escrito académico técnico (y por calco del lenguaje académico que se nutre del inglés), se denomina estado del Arte a la base teórica sobre la que se sustenta el escrito, o la cual se rebate en el desarrollo posterior en el escrito y que forma parte introductoria del mismo. Es frecuente que el segundo capítulo de una tesis doctoral en ingenierías se denomine "Estado del arte" donde se hace un repaso de las técnicas relacionadas con dicha tesis doctoral. Este capítulo es fundamental para explicar los aportes al conocimiento que realiza la tesis al estado del conocimiento actual.
El origen de la expresión, muy probablemente, se debe a Aristóteles en el libro primero de Metafísica, donde clasifica el conocimiento en ciencia, arte y experiencia.

- La ciencia busca el conocimiento por la mera curiosidad innata del ser humano. De hecho el libro primero de Metafísica de Aristóteles comienza con su famosa frase «Todos los hombres tienen por naturaleza el deseo de saber» (Elcho Pan, 1988:45). La ciencia no busca ninguna utilidad en la búsqueda del conocimiento, sino satisfacer la propia curiosidad innata del ser humano. Así pues son ciencias la filosofía, las matemáticas, la física, la biología, la química, etc.
- El arte, a diferencia de la ciencia, busca una utilidad a la búsqueda del nuevo conocimiento, cómo hacer mejores puentes con un menor coste de materiales y fabricación, cómo curar ciertas enfermedades o cómo poner un hombre en la luna. Así pues, según la clasificación aristotélica, arte es la medicina, la arquitectura, la informática y el resto de ingenierías, pero también la música, la poesía, la pintura, etc. Sin embargo, son estas últimas las que se han adueñado en la actualidad del término y son las que todo el mundo entiende por arte.
- Finalmente, Aristóteles se refiere a la experiencia como el conocimiento que poseen los diferentes oficios, pescador, agricultor, carpintero, etc.

Esto explica que en las tesis doctorales realizadas en ciencias, dicho capítulo se llame Estado de la Ciencia y en ingenierías en general Estado del arte.

- Solvers satisfactibles insatisfactibles
- que es SAT
- regresión lineal
- microservicios
- hilos y concurrencia
- beneficios de un sistema distribuido

3 Análisis de requisitos

En la Ingeniería de software, un Análisis de Requerimientos es una tarea que cubre el hueco entre la definición del software a nivel sistema y el diseño del mismo.

4 Definición y desarrollo

4.1 Planificación temporal del trabajo

El trabajo puede ser dividido en las siguientes partes:

1. Definición del sistema
   - Definir las responsabilidades que tienen tanto los nodos como los servicios.
   - Concretar que debe de especificar la definición de un servicio, como se obtiene el identificador del mismo y el tipo de serialización que se debe de usar.
2. Primera fase de desarrollo
   - Desarrollar las funciones básicas de un nodo:
     - Compilar a la especificación de un servicio.
     - Construir la especificación de un servicio.
       Dada la especificación de un servicio, debe ser capaz de construir un contenedor del mismo, abrir los puertos necesarios y añadirle las variables de entorno dadas, si es el caso.
     - Servicio local con la finalidad de atender las peticiones de los servicios.
       Las peticiones básicas son: la inicialización de un servicio, y su detención. Al especificarle un servicio, el servicio local del nodo le devolverá al cliente la dirección de la instancia del servicio demandado y un token con el que efectuar acciones u obtener información sobre él (como detenerlo, por ejemplo).
   - Desarrollar servicios SAT-solver y sus especificaciones.
3. Segunda fase de desarrollo
   - Desarrollar un servicio de clasificación de SAT-solvers.
     Por un lado se ha de desarrollar un sistema que genere CNFs aleatorios, pruebe cada uno de los solvers y recoja datos de ello. A partir de los datos obtenidos previamente se deben clasificar los solvers mediante algun tipo de regresión.
     Por otra parte, el servicio debe de tener la capacidad de, a partir de un CNF dado, resolverlo dando uso del solver que mejor se adapte a ese CNF en concreto.

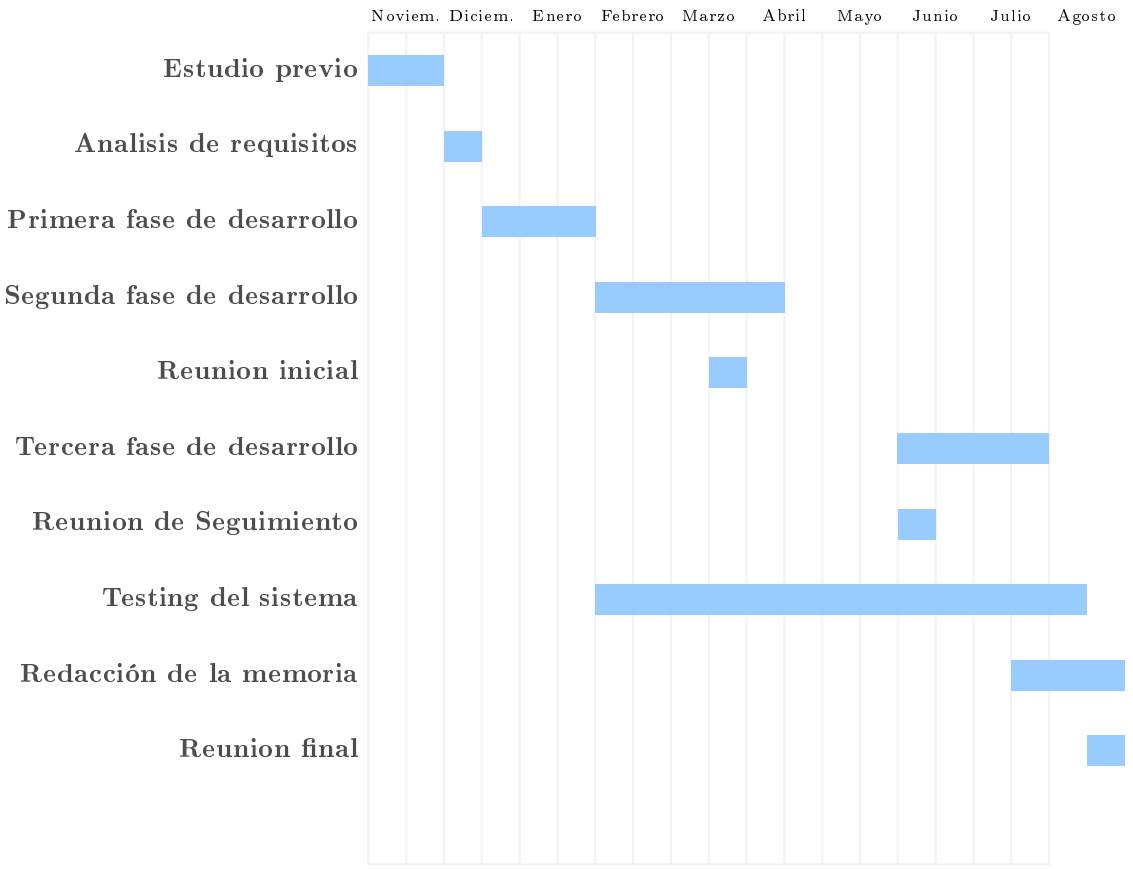
4. Tercera fase de desarrollo

   - Balanceo de carga de instancias.
     Al ser el nodo el que tiene la responsabilidad de construir y ejecutar los servicios demandados, este puede derivar a otros nodos de la red la tarea de construir y ejecutar un servicio.
     Se debe de desarrollar la lógica necesaria para que lo haga.
   - Balanceo de carga de solicitudes.
     El clasificador deberá tener un balanceador de carga con el que se repartirán las peticiones para resolver CNF's entre varias instancias de un mismo solver.
   - Distribución de contenedores.

   - Busqueda de otro nodos en la red.

5. Testing del sistema
   Tras cada fase de desarrollo.

Diagrama de Gantt:



4.2 Definición del sistema

Los conceptos principales que se han definido son:

- Un servicio es un contenedor de software que realiza una tarea especifica, puede entenderse como una "caja negra", existe una especificación del mismo en formato binario, donde se

especifica todo lo necesario para construirlo y comunicarse con él que hace innecesario conocer su funcionamiento interno.

- Un nodo es un computador que forma parte de la red, debe de tener la capacidad de encontrar otros nodos y comunicarse con ellos, compartir y construir servicios y orquestar los servicios entre él y otros nodos.
- El término instancia se refiere a la instancia de un determinado servicio, es decir, un contendor que ejecuta siguiendo los critérios de la especificación de dicho servicio.
- Tanto los servicios como los nodos ejecutarán servidores con los que atender solicitudes, concretamente en el proyecto servidores Grpc.
- La especificación de un servicio se refiere a un buffer de bytes que define el servicio, esto es, como esta formado y como comunicarse con él, y no se refiere a como construirlo. La especificación de un servico se serializa en un binario Protobuf.
- La imágen de un servicio se refiere a la imágen del contenedor de un servicio, asi como un contendor se refiere a la instancia de la imagen de un servicio.
- El concepto implementación del nodo pretende remarcar aquellas decisiones sobre la lógica del nodo que son independientes del funcionamiento general de la red y, por tanto, otros nodos pueden no adoptar.
- La configuración de un servicio se refiere a las variables de entorno usadas con este. Sirve para identificar una 'versión' concreta de un mismo servicio.
- Una instancia zombie es aquella cuyo servicio padre, quien la solicitó, ha perdido su token y por tanto no puede matarla, o bien el servicio que la solicito ya ha muerto.

Definir como debe comportarse un sistema requiere abordar cuales son los deberes y derechos de sus componentes.
Un nodo debe de proporcionar una interfaz con la que los servicios puedan comunicarse, junto con la dirección a la que el servicio le realizará sus solicitudes. La forma en la que el nodo carga estos componentes en una nueva instancia es mediante un archivo binario en la raíz del sistema de ficheros del servicio (__config__). Esto es así ya que el servicio podría requerir de estos datos antes de levantar su servidor.
Ante la solicitud de una nueva instancia, un nodo tiene el deber de, o bien construir esta instancia localmente, o bien solicitarle la instancia a un par (otro nodo), y retornar al solicitante tanto una dirección con la que comunicarse con la nueva instancia como un token con el que realizar acciones sobre ella. En caso de crear la dependencia de un servicio en otro nodo, el nodo debe de asegurarse que el solicitante tenga acceso a la dirección que le otorga para comunicarse con su dependencia.
Esta arquitectura permite a un servicio no preocuparse del nodo donde se ejecutará cada dependencia. Del mismo modo que los nodos no se preocuparán de la utilidad que puedan tener las instancias que ejecutan.
Si un nodo posee una función con la que obtener el coste de ejecutar la instancia de un determiando servicio. Sin especificar en que debe de basarse este coste.

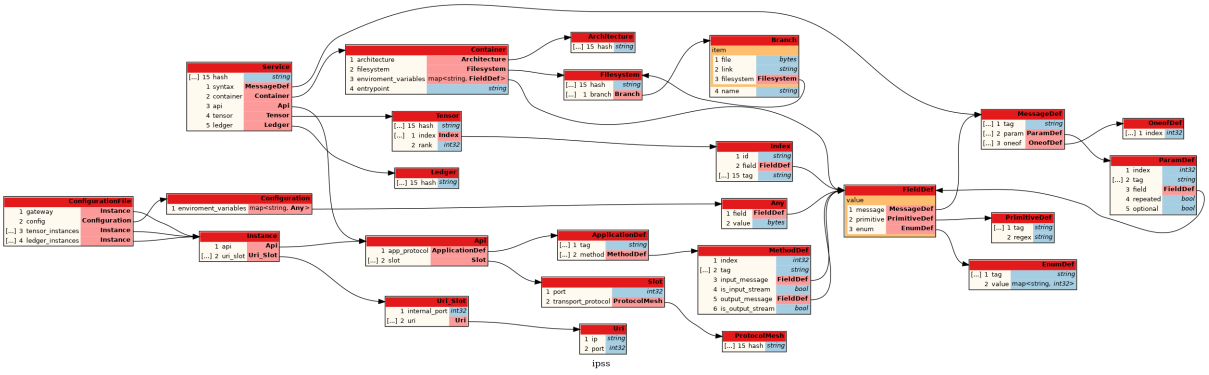
Por tanto, para resumir, durante la construcción de la instancia de un servicio, el nodo añade en su raíz un binario que contiene como comunicarse con él. Si el servicio quiere dar uso de una dependencia, le enviará la especificación de ésta al nodo, y éste le devolverá una dirección con la que comunicarse y un token para poder matarla en un futuro. Será el nodo el que busque donde

construir esa dependencia, si localmente o en alguno de sus pares. También será el nodo el que, en caso de matar a la primera instancia, acabe con la segunda (ya que es dependencia de la primera).

La parte más extensa de la definición del sistema es la especificación de un servicio, este se compone de los siguientes atributos:

- hash: Lista de hashes (del tipo 'sha256':913ref3...) sobre el binario del resto de la especificación.
- Syntax: Sintaxis de la especificación.
- Container: Especifica el contenedor donde se ejecuta el servicio, esto es:
  - la arquitectura, define el hardware y el kernel donde se debe de ejecutar el servicio.
  - el punto de entrada, define la ruta del primer ejecutable del servicio.
  - el sistema de archivos, representado en un árbol donde las ramas pueden ser o bien un archivo binario, o un link a otro elemento del árbol, o otro árbol, y donde cada elemento se identifica con un nombre.
  - las variables de entorno, lista compuesta por llave y definición del campo de cada variable.

  No abarca como se debe de construir (Dockerfile, capas de Docker ...), así cada nodo puede hacerlo de manera distinta.
- Api: Define cómo se puede interactuar con el servicio, constituida por:
  - interfaz de la aplicación (mensajes y métodos junto con errores y respuestas de cada uno).
  - lista de slots. El motivo de requerir slots es el de permitir comunicarse con un servicio en distintos protocolos. Cada slot está compuesto por los siguentes elementos:
    * puerto donde escucha (desde el servicio, de solicitarselo a otro nodo éste podría dar otro).
    * protocolo de comunicación que soporta (HTTP/2, GRPC ...).
- Tensor: Opcional. Definie la topología del modelo que entrena el servicio. Un servicio puede solicitar al nodo que le retorne las instancias de aquellos servicios que tengan un tensor determinado y con ellas realizar solicitudes a servicios que ya tienen el modelo entrenado.



Despues de especifiar los componentes del sistema y a que se dedica cada uno se ha realizado una implementación de cada uno de ellos para materializar la descripción de este.

4.3 Primera fase de desarrollo

La primera fase de desarrollo abarca la lógica detrás de un nodo, esto es, la compilación y construcción de servicios y el manejo de solicitudes de los mismos. El entorno donde se ejecuta esta implementación del nodo será Python3 sobre Ubuntu(Linux), dando uso, además, del sistema de base de datos MongoDB, donde se almacenan la información relativa a otros nodos de la red, y el motor de contenerización DockerEngine, con el que ejecutar instancias de servicios. El uso obligatorio de Linux se debe principalmente a que dentro de la lógica implementada en Python se da uso de comandos Bash. Además de eso, se da uso de Systemd para correr tres demonios de manera intermitente e indefinida: docker, mongo y gateway.py. La lógica detras del nodo se puede dividir en tres ficheros principales: compile.py, build.py y gateway.py.

Existen, además, dos directorios, registry y hycache, en registry se almacenan las descripciones de servicios provenientes, o bien de su compilación (compile.py) o bien una solicitud de otro servicio interno o externo (gateway.py), y el directorio hycache sirve para almacenar datos durante diferentes operaciones, ya sea para almacenar el contenedor comprimido de un servico al ser solicitado a otro nodo (build.py), guardar la configuracion de una nueva instancia para copiarla dentro de su contenedor (gateway.py) o para almacenar y calcular la hash de un sistema de archivos durante la compilación de un servicio (compile.py).

Una vez definidos los componentes principales del nodo se definen más en detalle cada una:

4.3.0.1 compile.py El objetivo de este script de Python es obtener la descripción de un servicio a partir de un directorio llamado .service/, ubicado en la raiz de un repositorio git, que contenga la información necesaria para ello, esto es: un archivo Dockerfile y distintos otra información contenida en un archivo Json.
Por lo tanto, desde la terminal de un nodo, el comando python3 /home/hy/node/compile.py <direccion del repositorio>::<rama del repositorio> generará la descripción del servicio en registry con nombre <sha256 del servicio>.service y la imagen del contendor en el registro de Docker con nombre <sha256 del servicio>.service.
El formato antes de la compilacion (Dockerfile y Json en un directorio .service en la raiz de un repositorio git) tiene como objetivo facilitar la compilación para un desarrollo que de uso de la herramienta Git, la definición del servicio es totalmente independiente de este, por lo que la compilación desde otro formato es facilmente implementable.
El sistema permite la compilación simultánea al generar un identificador aleatorio por cada ejecución, de esta forma el almacenamiento de datos en hycache ocurre dentro de un subdirectorio distinto por cada una. El script ejecuta comandos de docker mediante bash, dando uso del método os.system de Python.

Al iniciarse la compilación se descarga el repositorio aportado en hycache, generar la descripción de un servicio requiere obtener sus atributos y calcular su hash. Gran parte de los atributos fueron almacenados en el archivo Json (Api, Envs, Entrypoint ...), lo único que se debe de hacer es leer el archivo Json y almacenarlos en un objeto Protobuf. Para el sistema de ficheros (que es un subatributo de container) será necesario construir su contenedor mediante el Dockerfile para obtener su sistema de ficheros (esto lo realiza el método parseFilesys), dar uso del método save de docker

y comprimirlo en un archivo .tar para posteriormente extraer el sistema de ficheros. Docker divide la imagen de un contendor por capas, aunque cumple su función al contenerizar, es indiferente a la hora de definir el servicio puesto que distintas combinaciones de capas son posibles para un mismo servicio. Es por ello que el archivo .tar donde Docker almacenó el contendor es en realidad un directorio con cada una de las capas comprimidas, para obtener el sistema de ficheros completo se debe de descomprimir cada una y adjuntar todas en el mismo directorio. Finalmente se añade el sistema de ficheros al objeto Protobuf.
Una vez esté el objeto Protobuf completo se puede proceder a calcular las hashes de este para, posteriormente, añadirlas al mismo. Esta implementación del nodo da uso de varios algoritmos hash, aunque es el sha3-256 el que toma como referencia para identificar la descripción y el contenedor en todo el sistema. Tras calcular las hashes se vacía el atributo referente al sistema de ficheros de la especificación del servicio (objeto Protobuf) para no hacerlo tan pesado, no estar completo supone que al compartirlo no será posible verificar sus hashes. Para finalizar la compilación el nodo guarda el objeto Protobuf, referente a la especificación del servicio, en registry y elimina los archivos pertinentes a la operación de hycache (subdirectorio con el identificador generado aleatoriamente al inicio).

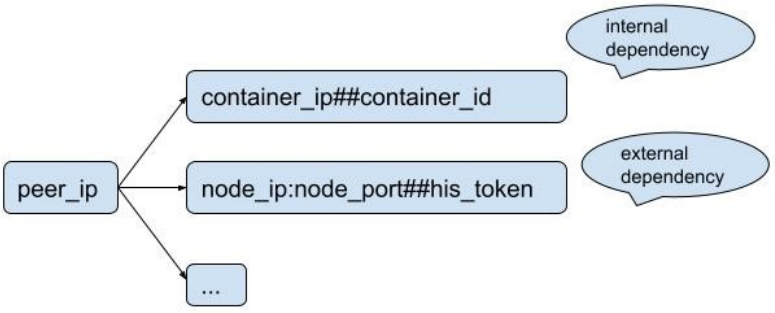
4.3.0.2 build.py Se compone de un único método (build) el cual recibe la especificación de un servicio (objeto ipss.Service) y construye la imágen docker para permitir generar instancias de este. Tomando la hash sha3-256 de la especificación del servicio (en la llamada al método se debe de asegurar que se encuentra esta hash, de no ser así el método no hace nada) verifica que se encuentre su respectiva imágen en el registro de Docker.

En caso de que no se encuentre la imágen se deberá buscar en otros nodos, por lo que se toman las instancias de otros nodos y dará uso del método GetServiceTar (supone que el resto de nodos lo implementan). El nodo de la red que posea el contenedor se lo devuelve mediante el streaming de lotes del archivo .tar del mismo, los cuales son almacenados en un archivo en hycache. Una vez que se ha descargado todo el archivo Docker se encarga de cargarlo en un contenedor y se le da el nombre correspondiente a su identificador (<hash sha3-256>.service).

Sería posible verificar la imagen obtenida de la red dando uso del método verify() ???

4.3.0.3 gateway.py Se trata del servidor Grpc encargado de atender las solicitudes tanto de las instancias que ejecuta el nodo como de otros nodos de la red. Su labor principal es ser capaz de generar instancias de servicios y destruirlas mientras gestiona de que instancia o nodo es dependiente cada una, esto es de vital importancia debido a que no puede fiarse del resto de agentes (nodos e instancias) por lo que si un nodo se desconecta o destruye una instancia deberá destruir todas sus dependencias. También es responsable de decidir si un determinado servicio se instancia localmente o solicita la instancia del servicio a otro nodo de la red (balanceo de instancias de servicios). Además de responder las solicitudes de otros nodos para obtener la imagen del contenedor de un servicio o su especificación y de presentarse a otros nodos de la red.

En esta primera fase de desarrollo solo se aborda la comunicación con instancias, esto es, crear dependencias y destruirlas.
Para ello las instancias se almacenan en Python3, dando uso de un diccionario seguro ante concurrencia con el objeto threading.Lock de Python, con el formato descrito a continuación:



Antes de explicar cada uno de los elementos de la imágen se debe de recalcar la diferencia entre dependencia interna y externa.

- una dependencia interna es aquella que el nodo decide ejecutar localmente.
- una dependencia externa es aquella que el nodo decide solicitar a otro nodo de la red.

Donde:

- peer_ip Se refiere al solicitante (servicio o nodo) de la instancia.
- container_ip Se refiere a la dirección ip que posee el contenedor de la instancia. Se da en caso de que la dependencia sea interna.
- container_id Se refiere al identificador que posee el contenedor de la instancia. Se da en caso de que la dependencia sea interna.
- node_ip:node_port Es la uri del nodo donde se ha solicitado la instancia. Se da en caso de que la dependencia sea externa.
- his_token Se refiere al token que posee la instancia en el nodo donde se le solicitó. Se da en caso de que la dependencia sea externa. Este formato del token varía entre unos nodos y otros, por lo que podría tener otro formato al que se describe acontinuación.

El token de una instancia posee el formato:

- peer_ip##container_ip##container_id en caso de una dependencia interna.
- peer_ip##node_ip:node_port##his_token en caso de una dependencia externa.

4.4 Segunda fase de desarrollo

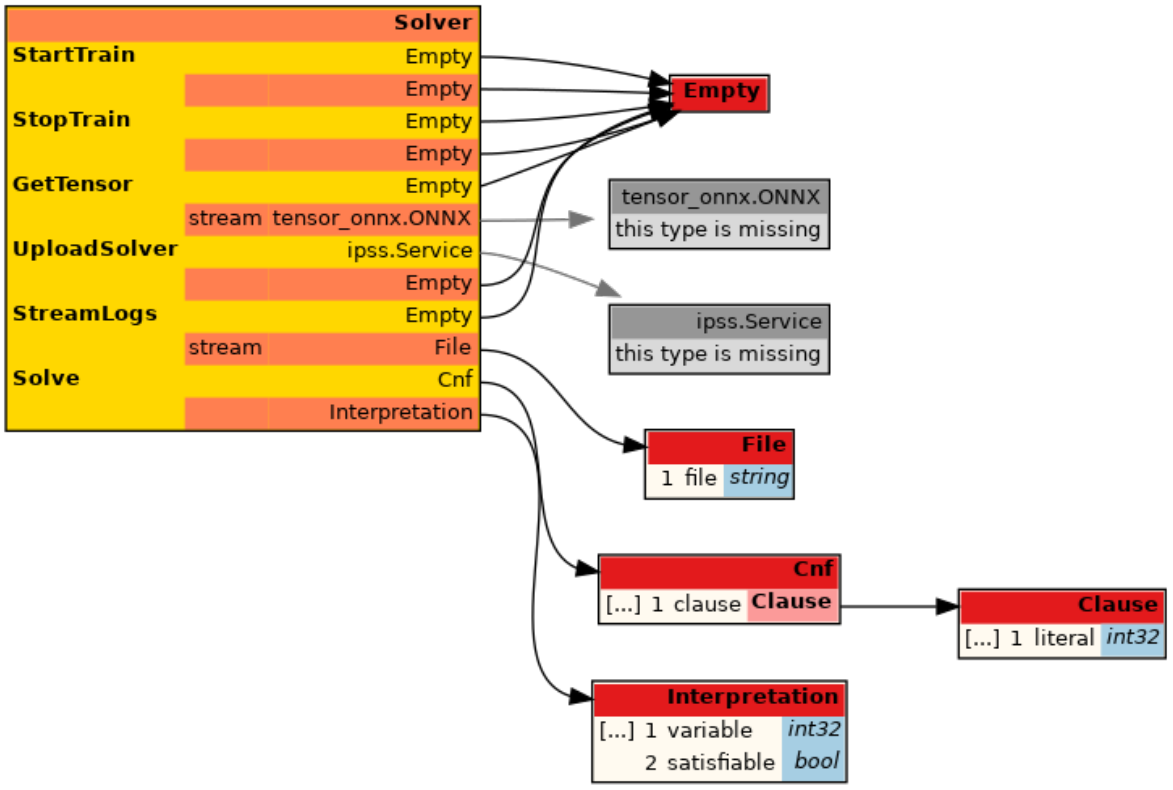
La segunda fase consiste en desarrollar el servicio de clasificación de solvers, este tiene la finalidad de: por un lado atender solicitudes para resolver cnf's, y por otro realizar un entrenamiento con cnf's aleatorios para clasificar diferentes servicios SATsolvers, esto con la intencion de utilizar el SATsorter más adecuado para cada cnf durante las solicitudes reales.

El servicio realiza varias tareas de manera simultanea, al igual que el nodo, se ha optado por dar uso de Python3 como tecnología sobre la que desarrollar su lógica. El punto de entrada del servicio

es el fichero start.py, donde se inicia el servidor Grpc tras acceder a la configuración inicial del servicio (archivo __config__ proporcionado por el nodo), tomar las variables de entorno e iniciar diferentes procesos secundarios.
Da uso de un servicio con el que obtener cnfs random, por lo que solicita una instancia de este en cada sesión de entrenamiento y la mata al finalizar la sesión. Además de esta dependencia también se le añade cada uno de los solvers, los cuales pueden ser añadidos por el cliente a través del servidor Grpc. El servicio de obtención de cnfs random no ha sido desarrollado durante el proyecto, solo adaptado, se da uso de un código del profesor Josep Argelich, por lo que su lógica no será desarrollada en la memoria, retorna un cnf de cualquier tipo, satisfactible o insatisfactible.

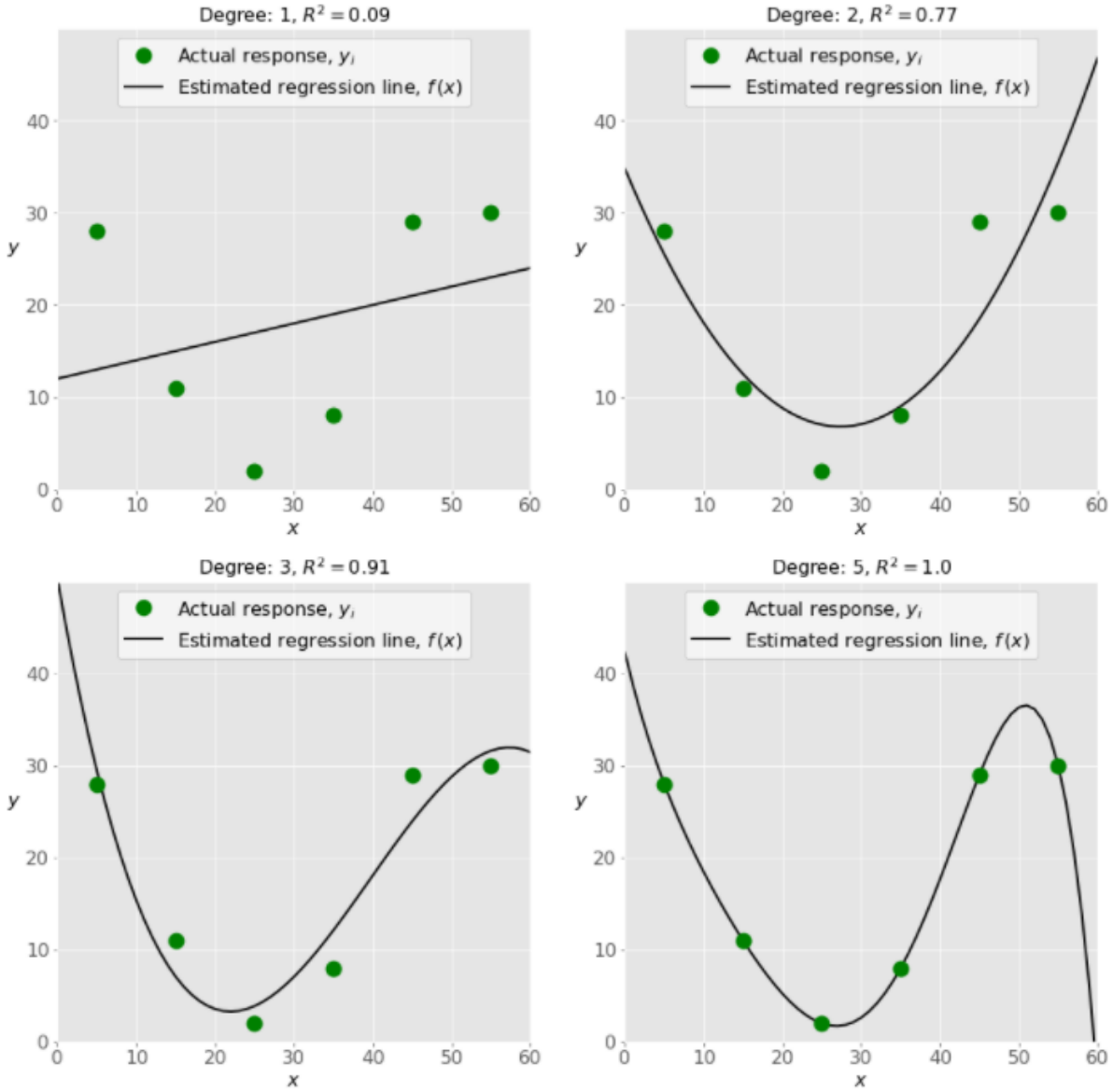
El servidor Grpc posee una serie de métodos con los que atender las solicitudes del cliente, estos son:

- StartTrain, inicia una sesion de entrenamiento si no está ya inicializada.
- StopTrain, si existe, detiene la sesión de entrenamiento.
- UploadSolver, recive la especificación de un servicio de resolución de SAT y la añade al sistema.
- StreamLogs, retorna los 'logs' del sistema mediante streaming.
- Solve, recive un cnf, calcula cual es el mejor solver con el que resolverlo y lo resuelve dando uso del mismo y finalmente retorna el resultado.



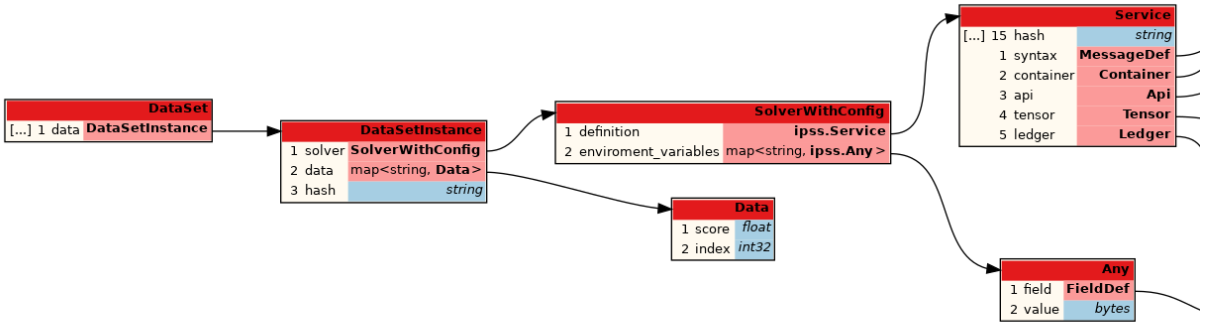
Las partes principales en las que se divide la lógica del servicio son:

4.4.0.1 regresion.py Se trata de un hilo independiente iniciado desde start.py antes de ejecutar el servidor Grpc. Se encarga de calcular la regresión lineal de cada set de datos enviado desde la sesión de entrenamiento lo almacena como tensor de Onnx en un fichero tensor.onnx. Si no esta la sesión de entrenamiento activa no se realiza la regresión porque no detecta cambios en el set de datos, la detección de cambios en el set de datos funciona usando un algoritmo hash sobre él.
Por lo tanto, cada cierto periodo de tiempo (TIME_FOR_EACH_REGRESSION_LOOP: 900 segundos por defecto si en la variable de entorno no se indica lo contrario) se ejecutará, en caso de haber habido cambios, la regresión de los datos recogidos por cada solver. La regresión de cada solver consiste en realidad en una regresión por cada grado hasta un máximo estipulado (MAX_DEGREE: 100 por defecto si en la variable de entorno no se indica lo contrario) y escoger aquella función que se ajuste más a los datos. De esta forma se realizan un total de <número de solvers> * MAX_DEGREE regresiones cada TIME_FOR_EACH_REGRESSION_LOOP segundos, siempre y cuando se realicen cambios en el set de datos.
El tensor posee el formato de una lista de tensores de Onnx (Intercambio de Redes Neuronales Abierto), un ecosistema de inteligencia artificial de código abierto, asociados cada uno a un SAT solver (especificación del servicio). Se da uso del estandar Onnx por ser un estandar abierto para compartir modelos de redes neuronales entre distintas librerías (TensorFlow o PyCharm), aunque en este caso se utiliza para compartir modelos de regresión lineal, y permite así que otro servicio de clasificación continue el modelo entrenado por el clasificador sin tener que utilizar la librería de la él da uso (Sklearn).



Se toma el modelo con el R2 más alto.

Sobre el set de datos del que da uso el proceso de regresión, la sesión de entrenamientos vuelca cada cierto tiempo los datos recogidos en un fichero (solvers_dataset.bin) al cual nos referimos como el set de datos. El formato del set de datos consiste en una puntuación por cada tipo de cnf (número de instancias * número de clausulas) por cada solver con una configuración determinada, cuya especificación se encuentra también en el set de datos.



4.4.0.2 train.py Durante el inicio del proceso principal (start.py) se obtiene el objeto trainer.Session, el cual sigue el patrón Singleton, este objeto es usado durante toda la vida del servicio.
El objeto train.Session se ocupa de:

- Gestionar el uso del servicio de obtención de cnfs random.
- ejecutar la sesión de entrenamiento.
- detener la sesión de entrenamiento.
- Recoger los datos de la sesión de entrenamiento en un dataset (solvers_dataset) y volcarla en el archivo solvers_dataset.bin cada cierto número de iteraciones (SAVE_TRAIN_DATA: 2 por defecto).

Cuando se inicia una sesión de entrenamiento (método start) desde el hilo principal se crea un hilo que ejecuta el método init, este inicia un servicio de cnfs random y, en cada iteración, genera un cnf y lo prueba en cada uno de los solvers. Solo se detiene con la señal de la variable do_stop.
Sobre el sistema de puntuación de los solvers, la puntuación que un solver recibe de la resolución de un cnf es inversamente proporcional al tiempo que el solver tarda en responder. En caso de que tarde en responder demasiado (existe un 'timeout' aplicado en la llamada Grpc de la variable de entorno TRAIN_SOLVERS_TIMEOUT, 30 segundos por defecto) se puntua en negativo (-1/time_out). En caso de que devuelva una interpretación se comprueba que esta sí satisface el cnf (operación con coste polinomial), si lo satisface puntua en positivo (+1/time) y registra que se demostro la satisfactibilidad del cnf, si no la satisface puntua en negativo (-1/time). En caso de que responda que el cnf es insatisfactible se guarda en una lista que ese solver tardo x tiempo en decir que era insatisfactible, al terminar de probar todos los solvers con ese cnf se comprueba si hubo alguno que demostrara su satisfactibilidad, de no ser asi, los solvers que lo dieron como insatisfactible puntuan en positivo (+1/time) y si se demostró la satisfactibilidad puntuan en negativo (-1/time).

Sobre la gestión del servicio de generación de cnfs random, se crea una instancia al iniciar cada sesión de entrenamiento y se destruye al finalizarla. Además de eso es necesario solicitar una nueva instancia en caso de que la instancia actual del servicio acumule una cantidad determinada de errores (CONNECTION_ERRORS: por defecto 5), originados al solicitar un nuevo cnf ya sea por un error de Grpc o porque salta su 'timeout'.

Sobre el mecanismo para detener el entrenamiento, hay que tener en cuenta que para evitar que la instrucción stop, usada por el método StopTrain del servidor Grpc en start.py, mate el hilo durante un entrenamiento el método stop emite el mensaje de que se debe de parar el entrenamiento en la siguiente vuelta, y a continuación espera a que el entrenamiento salga del bucle y termine. De esta forma se asegura que no deja instancias fuera de pila y por tanto servicios zombie (la pila se aborda en la fase tres).

4.4.0.3 _solve.py Durante el inicio del proceso principal (start.py) se obtiene el objeto solve.Session, el cual sigue el patrón Singleton, este objeto es usado durante toda la vida del servicio.
El objeto solve.Session se ocupa de:

- gestionar las instancias de todos los solvers
- solicitar a un solver determinado que resuelva un cnf determinado
- balancear las solicitudes entre las distintas instancias de un mismo solver en caso de ser necesario (este punto se aborda durante la tercera fase de desarrollo).

Al solicitar el objeto solve.Session por primera vez se iniciará un hilo dedicado a mantener las instancias de los solvers denomidado Maintainer. Este comprueba, cada cierto periodo de tiempo (MAINTENANCE_SLEEP_TIME: 100 segundos por defecto), si las instancias siguen activas resolviendo un cnf básico ([[1,],]), en caso de obtener un error se encarga de matar la instancia.

El objeto solve.Session también posee un método con el que añadir nuevos solvers a su registro, estos se almacenan en un diccionario del tipo identificador:SolverConfig. El objeto SolverConfig detalla la definición de un servicio junto con una configuración del mismo determinada.

Finalmente, el objeto solve.Session permite resolver cnfs dado un cnf, el identificador de un solver con una configuración determinada, y con la posibilidad de especificar el timeout de la operación y proporcionar la especificación del solver junto con su configuración, por si con el identificador no se encuentra. En caso de que ese solver se encontrara en el registro, se resuelve el cnf y se retorna la respuesta junto con el tiempo que tomó resolverla. En caso de que no se encuentre y se halla proporcionado una especificación la añadirá al registro y retornará error.

El manejo y control de las instancias de cada solver se desarrolla más adelante, en la tercera fase, puesto que su arquitectura se ve fuertemente influenciada por la necesidad de permitir varias instancias por solver.

4.4.0.4 _get.py Trara de seleccionar el solver que mejor rendimiento va a dar para un cnf determinado. Carga el tensor del archivo tensor.onnx y realiza inferencia sobre el modelo de regresión lineal de cada uno de los solvers contenidos en tensor.onnx.
La inferencia es el proceso por el cual se derivan conclusiones a partir de premisas, es decir, a partir de un modelo de regresión lineal se supone la puntuación que se obtendría para un cnf determinado. Para la inferencia se da uso de la libreria ONNX-Runtime de Microsoft.

El patrón singleton o instancia única es un patrón de diseño que permite restringir la creación de objetos pertenecientes a una clase o el valor de un tipo a un único objeto. De esta forma se limita a que solo pueda haber una instancia de las clases train.Session y solve.Session.

4.5 Tercera fase de desarrollo

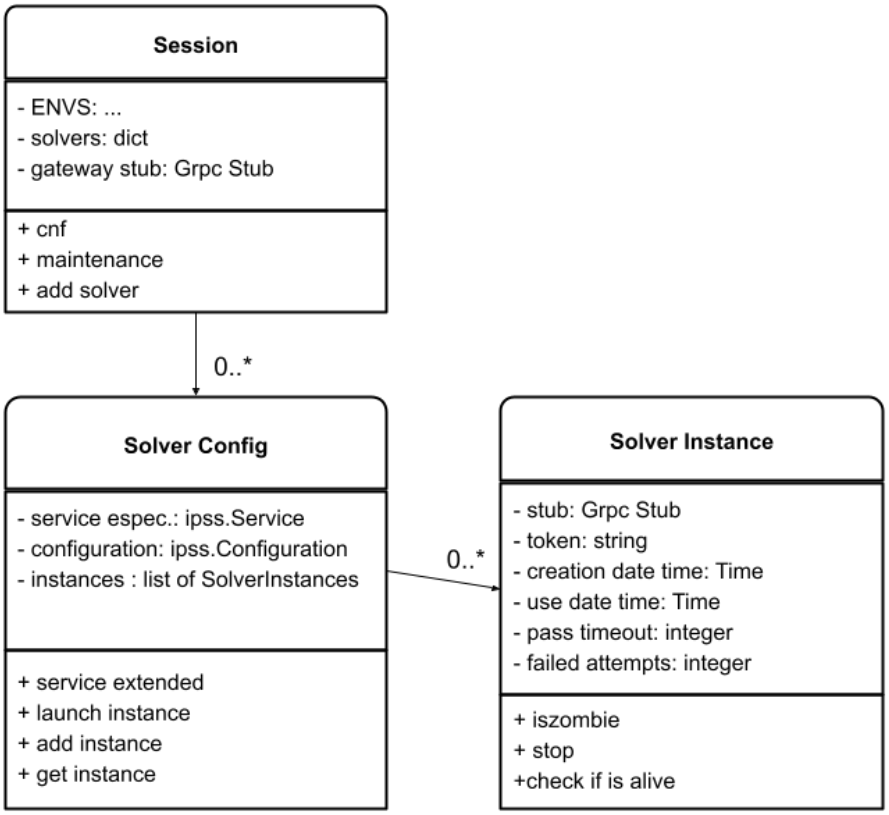
La tercera fase cubre tres desarrollos paralelos:

- el balanceo de carga de instancias en el nodo.
- el balanceo de carga de solicitudes en el clasificador.
- la distribución de contenedores en el nodo.
- añadir nuevos nodos de la red en el nodo.

4.5.0.1 Balanceo de instancias Cuando el nodo recibe una solicitud para instanciar un servicio es este el que decidirá en que nodo de la red se debe de instanciar, localmente o en otro nodo. Para ello compara el coste de ejecución de la instancia en cada uno de los nodos que posee el registro (MongoDB) con el número de contenedores que se están ejecutando en ese momento y escoge el de menor coste. Ese mismo coste local, el número de contenedores que se están ejecutando, es el que retornará a otros métodos cuando le pregunten por el coste de ejecutar una instancia (método Grpc GetServiceCost), sin importar que instancia sea.

4.5.0.2 Balanceo de solicitudes En el supuesto de que el cliente realiza, al mismo tiempo, multitud de solicitudes para resolver solvers, podría darse el caso de que, en un mismo instante de tiempo, se requiera dar uso de un solver en concreto. El solver en cuestion podría ser capaz de resolver varios cnfs simultaneamente, dentro del mismo contenedor, aunque de esa forma no se aprobechan las ventajas del sistema distribuido. Con el objetivo de que un varios cnfs se resuelvan simultaneamente por un mismo solver el clasificador crea distintas instancias de este cuando es necesario, de esta forma el nodo se encargará de instanciar el servicio entre los distintos nodos de la red.

Antes de entrar en detalle sobre la estructura sobre la que se manejan las instancias se debe de exponer el diagrama de clases siguiente:



Como se puede observar en el diagrama anterior, para manejar las instancias de cada solver, se adopta una estructura de un diccionario de solvers (<solver config. id>:<solver config objet>), donde cada objeto SolverConfig posee una lista de instancias (objeto SolverInstance). Cuando se requiere la instancia de un solver determinado se quita de la lista y cuando se deja de usar se vuelve a introducir en la lista. El acceso a todo el diccionario se proteje contra la concurrencia mediante el uso de un objeto threading.Lock.

Hay dos métodos que obtienen e introducen instancias en las listas, resolver un cnf (Session.cnf) y el del hilo de mantenimiento (Session.mainteance).
El método para resolver un cnf da uso de la lista como si se tratara de una pila normal, es decir, toma el elemento que se encuentra en el índice más elevado de la lista y coloca el elemento en el índice más elevado de la lista.
El método de mantenimiento da uso de la lista como si se tratara de una pila inversa, es decir, toma el elemento que se encuentra en la primera posición de la lista y coloca el elemento en la primera posición también.
El motivo de adoptar esta configuración es que las instancias tienen contadores sobre el último tiempo en que se usaron, estos solo se actualizan al solicitar resolver cnfs (el método Session.cnf),

por lo que cuando el mantenedor de instancias la toma comprueba si se paso de tiempo en deshuso, si es asi no lo vuelve a cargar en la lista, de lo contrario la deja en primera posición.
Si se toma una instancia, se debe de asegurar que al terminar de usarla, o bien se agrega a su lista correspondiente, o bien se para. No asegurar esto ocasiona un bug importante ya que la instancia quedaría en estado zombie hasta que el clasificador fuera eliminado (en el supuesto en que el nodo donde se instancio matara dependencias que fueran a quedar zombies).

4.5.0.3 Distribución de contenedores Con el fin de atender las solicitudes de obtención de un contenedor el nodo proporciona un método Grpc (método Grpc GetServiceTar), este recive la especificación de un servicio y, si puede proporcionarlo, comprime el contenedor y envía el archivo en formato .tar dividiendolo en trozos y enviandolos en streaming, en caso de no poder proporconar el contenedor salta una Excepción que recive el cliente.
Para la busqueda de contenedores, el nodo solo debe de iterar entre sus pares (resto de nodos de la red) y comprobar si alguno posee el contenedor dando uso del método anteriormente definido.

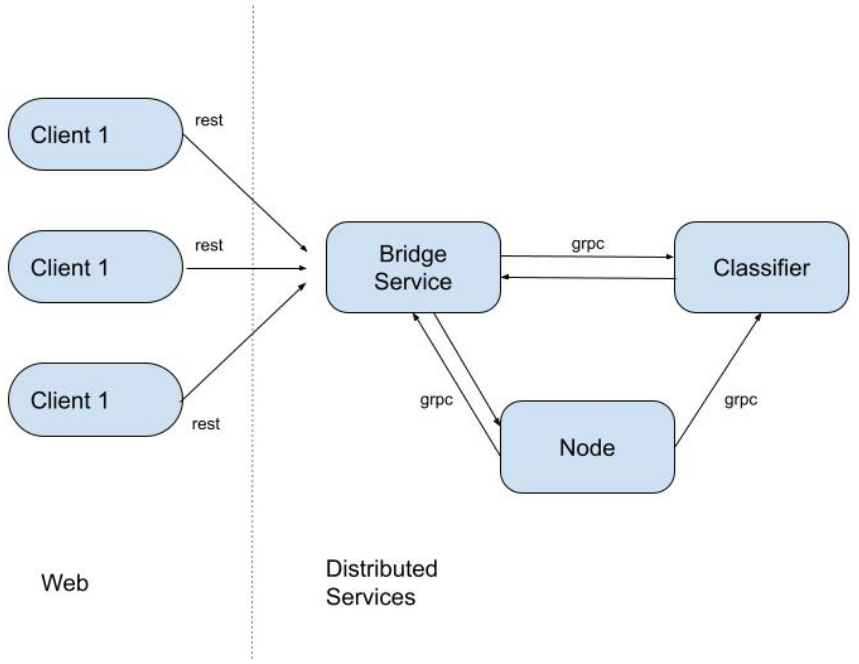
Para la distribución de especificaciones de servicios se utilizan métodos Grpc con mecánicas similares.

4.5.0.4 Conexión con nuevos nodos Al añadir un nuevo nodo a la red se debe de usar el comando 'python3 zeroconf.py <uri del nuevo nodo>' desde la consola de cada nodo, el nodo donde se ejecuta el comando ejecutará el método Grpc HyNode en el supuesto nuevo nodo, si este le responde con un objeto Instance, el nodo le retornará su instancia y lo tomará como un nodo completamente operativo, si no responde olvidará la solicitud.

5 Problemas encontrados y decisiones técnicas tomadas

5.1 ¿Cómo realizan múltiples clientes solicitudes al servicio de clasificación?

Basándose en la arquitectura anteriormente desarrollada, si un cliente quiere dar uso de un servicio, éste solicita la creación de una instancia del mismo a su nodo, por lo tanto, si n clientes quieren dar uso del servicio, serán necesarias n instancias de éste, a fin de cuentas, la instancia de un servicio, un contenedor, no es más que un hilo aislado. Por lo tanto, el paradigma cliente/servidor se ve alterado al hablar de servicios distribuidos. En vez de un servidor atendiendo múltiples clientes, un cliente solicita el despliegue de un servicio. En términos de rendimiento no debería existir una gran diferencia entre un esquema y otro, puesto que el servidor centralizado termina ejecutando n hilos para n solicitudes de clientes, en los servicios distribuidos estos hilos se aislan (esto puede ser un punto negativo sobre el uso de memoria compartida) y se reparten entre los nodos de la red.
Ahora bien, en caso de querer realizar un servicio web, al que poder, tal vez, conectar con un frontend en un navegador, será necesario un nuevo servicio el cual se encargue de recibir las solicitudes de los clientes y desplegar y dar uso del servicio de clasificación. Este servicio por tanto, puede ser comprendido como un puente entre la "web tradicional" y una red de servicios distribuidos.



5.2 Varias instancias no pueden compartir dependencias

La naturaleza que adopta la red crea una jerarquía de instancias de servicios repartidas entre los nodos de la mísma.
Esto significa que, cuando una instancia (padre) solicita una instancia de otro servicio (hijo), este último pasará a ser dependiente del primero, adoptando el padre el derecho a decidir cuando debe de morir el hijo y la responsabilidad sobre su consumo de recursos (en caso en que un nodo realice un seguimiento del consumo de cada instancia, deberá tener en cuenta el de sus dependencias). De esta forma, si se mata al padre, el nodo donde se este ejecutando su hijo podrá matarlo.
Si no fuera asi, un nodo no sabría que instancias quedan en estado 'zombie' ya que ninguno de sus clientes (instancias padre) se haría responsable.

5.3 Json vs GRPC

Json se ha convertido en prácticamente un estándar a la hora de compartir información en la web, debido a que es el sistema de serialización de Javascript, y éste, a su vez, es el lenguaje estandarizado para ejecutarse en los navegadores web.

Pero en una red de microservicios, donde los entornos de ejecución son virtualizados, Javascript pierde protagonismo y, por tanto, Json también. Es por esto que cabe preguntarse cual es la mejor forma de serializar las comunicaciones entre servicios. Al ser evidente que no existe un protocolo capaz de liderar ni en cualquier ámbito ni indefinidamente, cada Slot de un servicio definirá el protocolo soportado.

Concretamente para este proyecto se dará uso de Grpc, principalmente por dos razones:

- Protobuf, su lenguaje de serialización, consigue comprimir bytes de manera muy eficiente enfocando los buffers a cada topología de mensajes.
- Utiliza Http/2, por lo que está preparado para realizar streaming bidireccional de manera eficiente.

Otros protocolos también a tener en cuenta podrían ser Flatbuffers o Capt'n proto.

5.4 ¿Porque contener todo el sistema de ficheros en la especificación de un servicio?

5.5 ¿Porque no añadir el Dockerfile en la especificación de un servicio?

5.6 Identificación del algoritmo hash

5.7 Sobre el mecanismo de coste de ejecución de instancias de servicios

5.8 Uso de cola en el mantenedor de instancias

6 Resultados y conclusiones

7 Trabajo futuro

References

[1] Deterministic and minimal docker images. https://gregoryszorc.com/blog/2014/10/13/deterministic-and-minimal-docker-images/.

[2] Documentation grpc. https://grpc.io/docs/.

[3] Envoy proxy. https://www.envoyproxy.io/.

[4] Flatbuffers documentation. https://google.github.io/flatbuffers.

[5] Gunicorn python. https://gunicorn.org/#docs.

[6] Ipfs documentation. https://docs.ipfs.io/.

[7] Isa-json format. https://isa-specs.readthedocs.io/en/latest/isajson.html.

[8] Open container initiative. https://opencontainers.org/.

[9] Open neural network exchange. https://onnx.ai/.

[10] Protocol buffers | google developers. https://developers.google.com/protocol-buffers.

[11] Python 3.9.3 documentation. https://docs.python.org/3/.

[12] Rest vs grpc. https://es.accentsconagua.com/articles/code/rest-vs-grpc-battle-of-the-apis.html.

[13] Scikit-learn.

[14] Sistemas con estado y sin estado. https://www.redhat.com/es/topics/cloud-native-apps/stateful-vs-stateless.

[15] threading - gestionar operaciones concurrentes dentro de un proceso. https://rico-schmidt.name/pymotw-3/threading/.

[16] Welcome to flask. https://flask.palletsprojects.com/en/1.1.x/.

[17] Bitcoin key generation. https://www.oreilly.com/library/view/mastering-bitcoin-2nd/9781491954379/ch04.html#:~:text=The%20bitcoin%20private%20key%20is,generated%20from%20the%20private%20key., Jun 2020.

[18] ¿qué es kubernetes? https://kubernetes.io/es/docs/concepts/overview/what-is-kubernetes/, Jun 2020.

[19] A beginner's guide to understanding and building docker images. https://jfrog.com/knowledge-base/a-beginners-guide-to-understanding-and-building-docker-images/, Mar 2021.

[20] Docker documentation. https://docs.docker.com/, Apr 2021.

[21] Documentacion ngnix. https://www.nginx.com/, Apr 2021.

[22] Ramanan Balakrishnan. A simplified guide to grpc in python. https://www.semantics3.com/blog/a-simplified-guide-to-grpc-in-python-6c4e25f0c506/, Jan 2019.

[23] Nigel Brown. Explaining docker image ids. https://windsock.io/explaining-docker-image-ids/, Apr 2018.

[24] Davi Frossard. Linear regression with numpy. https://www.cs.toronto.edu/~frossard/post/linear_regression/, May 2016.

[25] Mano lingam. Hashes are unique. https://medium.com/predict/hashes-are-unique-9af14fe3796a, Apr 2019.

[26] John Minnihan. Understanding the docker cache for faster builds. https://thenewstack.io/understanding-the-docker-cache-for-faster-builds/, Mar 2021.

[27] Real Python. Linear regression in python. https://realpython.com/linear-regression-in-python/, Mar 2021.